

PROJECT REPORT

Project Title: FitBuddy - AI Workout Generator

Program: Naan Mudhalvan (SmartBridge)

1. Team Details

- **Team ID:** SWTID-2026-2770
- **Team Size:** 4
- **Team Leader:** Manikandaprabu
- **Team Members:**
 - Muthuvelan R
 - Mohamed Kanifa P
 - Syed Sulaiman

2. Abstract

FitBuddy is a personalized AI-driven fitness planner designed to dynamically generate customized 7-day workout plans and nutrition tips. Recognizing that standard fitness plans do not work for everyone, this application leverages Generative AI to tailor workout routines based on an individual's age, weight, specific fitness goals (e.g., weight loss), and preferred intensity levels. Built on a fast and modern web framework, FitBuddy provides a seamless user experience from data input to real-time AI generation and database storage.

3. Project Objectives

- To develop a responsive web application that collects user fitness profiles and goals.
- To integrate Google Gemini Generative AI to create customized, day-by-day workout plans instantly.
- To store user data and generated plans persistently using a relational database for future retrieval and admin monitoring.
- To implement seamless fallback mechanisms (timeout handling) ensuring a continuous user experience even during API latency.

4. Technology Stack

- **Programming Language:** Python 3
- **Backend Framework:** FastAPI
- **Database & ORM:** SQLite, SQLAlchemy
- **AI Integration:** Google Gemini API (gemini-3.5-flash-lite)
- **Frontend:** Jinja2 Templates, HTML, CSS

- **Server:** Uvicorn

5. System Architecture & Flowchart

The application follows a linear client-server architecture with external API integration:

1. **User Input:** The user submits their details (Name, Age, Weight, Goal, Intensity) via the Jinja2 HTML form.
2. **FastAPI Route:** The backend receives the POST request and temporarily stores the data.
3. **Gemini API Processing:** A crafted prompt containing the user's data is sent to the Google Gemini API.
4. **Response Generation:** The AI returns a structured 7-day plan and a separate nutrition tip.
5. **Database Storage:** SQLAlchemy commits the user profile and the original AI-generated plan into the SQLite database.
6. **Jinja2 UI:** The final customized plan is rendered elegantly back to the user on the result page.

Data Flow: User UI → FastAPI (routes.py) → Gemini API → SQLite (database.py) → Result UI

6. Key Code Snippets

A. Database Schema (database.py)

Python

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer)
    weight = Column(Float)
    goal = Column(String)
    intensity = Column(String)

class WorkoutPlan(Base):
    __tablename__ = "workout_plans"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer)
    original_plan = Column(String)
```

B. Gemini AI Integration (gemini_generator.py)

Python

```
import google.generativeai as genai
import os

genai.configure(api_key=os.environ.get("GOOGLE_API_KEY"))
model = genai.GenerativeModel("gemini-3.5-flash-lite")

def generate_workout_gemini(user_input):
    prompt = f"Create a structured 7-day workout plan for {user_input['goal']} at {user_input['intensity']} intensity."
    try:
        response = model.generate_content(prompt,
request_options={"timeout": 60})
        return response.text
    except Exception as e:
        return f"Error: {e}"
```

C. FastAPI Routing (routes.py)

Python

```
@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(request: Request, user_id: int = Form(...), goal: str
= Form(...), intensity: str = Form(...)):
    # Database Logic
    db = SessionLocal()
    # AI Generation
    user_input = {"goal": goal, "intensity": intensity}
    plan = generate_workout_gemini(user_input)
    # Save & Render
    return templates.TemplateResponse(request=request, name="result.html",
context={"workout_plan": plan})
```

7. Output Screenshots

(Note: Paste your screenshots below this line)

• **Screenshot 1: Home Page / User Input Form**

FitBuddy - AI Workout Generator

Name:

User ID:

Age:

Weight (kg):

Fitness Goal:

Workout Intensity:

[Generate Plan](#)

[Go to Admin Dashboard](#)

• **Screenshot 2: Generated 7-Day Workout Plan**

Your Personalized Workout Plan

User Information

Name: I.Manikanda Prabu | User ID: 102
Age: 21 | Weight: 69.0 kg
Goal: Weight Loss | Intensity: Medium

Workout Plan

Welcome to your 7-day personalized weight-loss plan! At a medium intensity, our goal is to keep your heart rate elevated, build lean muscle to boost your metabolism, and ensure you're recovering properly to stay consistent. Let's get to work!

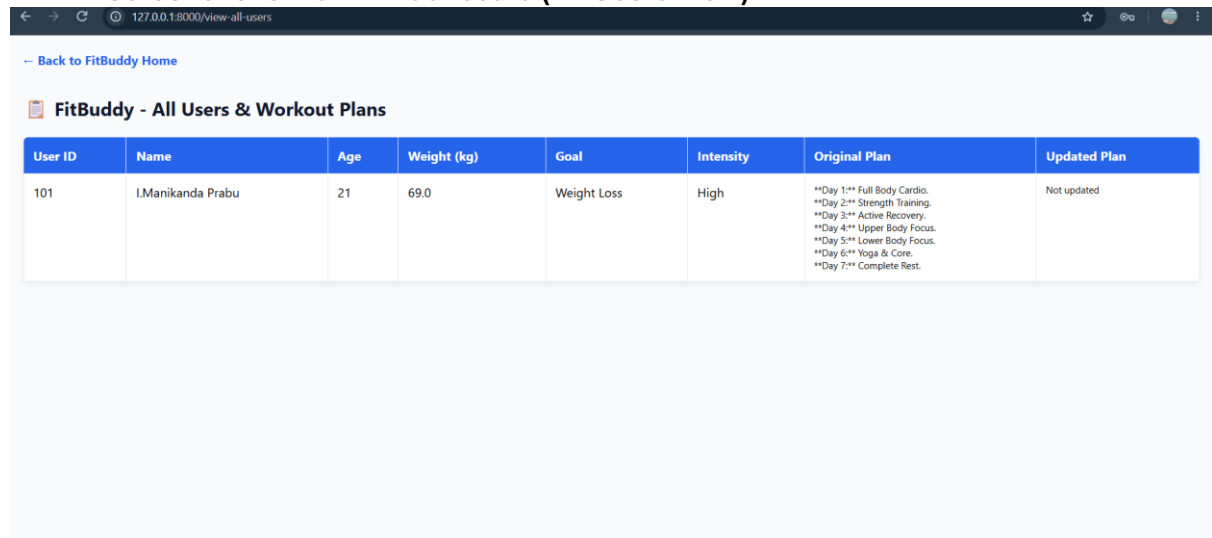
****Day 1: Full Body Strength & Cardio****

Warm-up: 5-8 minutes of dynamic movements (arm circles, torso twists, high knees, and jumping jacks to get the blood flowing).

Main Workout: 3 sets of each exercise, 45 seconds of work followed by 15 seconds of rest:

- Bodyweight Squats
- Push-ups (modify on knees if needed)
- Dumbbell or Kettlebell Deadlifts (moderate weight)
- Plank shoulder taps
- Alternating reverse lunges

• Screenshot 3: Admin Dashboard (All Users View)



The screenshot displays the 'FitBuddy - All Users & Workout Plans' page. It features a table with columns for User ID, Name, Age, Weight (kg), Goal, Intensity, Original Plan, and Updated Plan. A single user is listed with a detailed 7-day workout plan.

← Back to FitBuddy Home							
 FitBuddy - All Users & Workout Plans							
User ID	Name	Age	Weight (kg)	Goal	Intensity	Original Plan	Updated Plan
101	I.Manikanda Prabu	21	69.0	Weight Loss	High	**Day 1:** Full Body Cardio. **Day 2:** Strength Training. **Day 3:** Active Recovery. **Day 4:** Upper Body Focus. **Day 5:** Lower Body Focus. **Day 6:** Yoga & Core. **Day 7:** Complete Rest.	Not updated

8. Conclusion

The FitBuddy project successfully demonstrates the integration of modern web frameworks (FastAPI) with advanced Generative AI (Google Gemini). By automating personalized fitness planning, the application serves as a scalable prototype for health-tech solutions. The project effectively met all Naan Mudhalvan requirements, showcasing robust backend routing, database management, and error handling.